DAY 4 OF 7

# Vue 3: Router, Vuex & login

A Vue app where users register and log in, a small Vuex store remembers who is logged in, and pages are protected by role

# Today's plan

| | |
|---|---|
| **10 min** | Recap: the API is ready |
| **20 min** | Create the project, tour of a .vue file |
| **15 min** | api.js: axios + interceptors |
| **20 min** | Vuex: one small store |
| **25 min** | Router + guard, login & register pages |
| **20 min** | Placeholder dashboards, demo |

### By the end you can

- Create a Vue 3 project with Vite
- Write components with the Options API
- Keep the logged-in user in a Vuex store
- Protect pages with a router guard and send the token with axios

# Single-page app

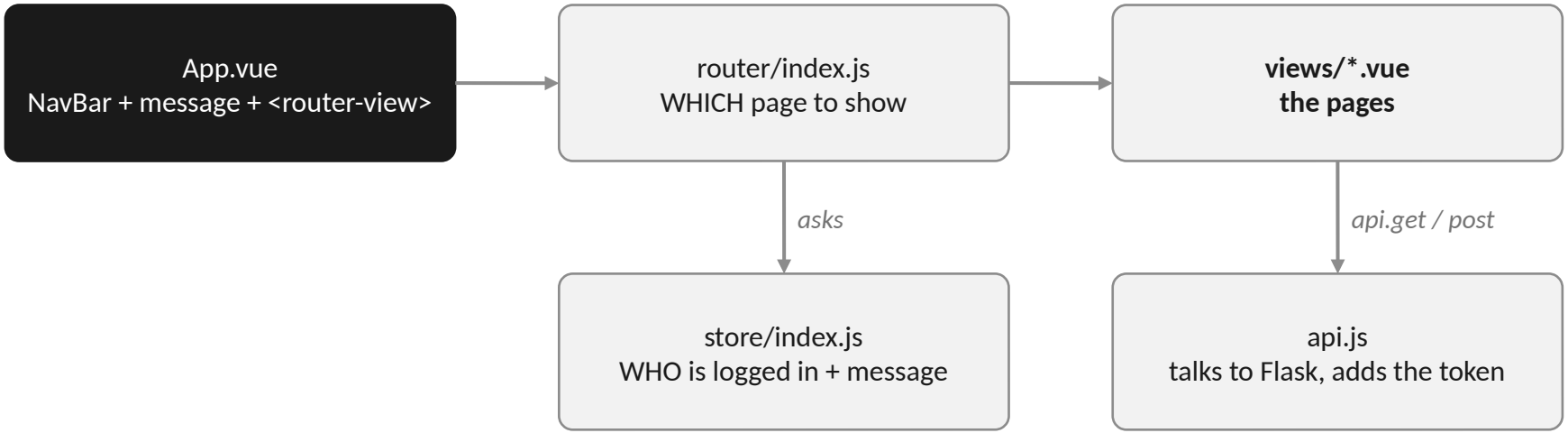
## Multi-page (MAD I)

- Every link loads a new HTML page
- Server builds each page
- State is lost on every page load

## Single-page (MAD II)

- One index.html, loaded once
- Vue swaps components when the URL changes
- Data comes from the API as JSON

# The frontend in one picture




Rule for the whole week: the store keeps only what many pages need. Each page loads its own data (subjects, quizzes…) with api.get in mounted().

# Create the project

```
Terminal

cd quiz-master
npm create vite@latest frontend \
    -- --template vue
cd frontend
npm install
npm install vue-router@4 vuex@4 \
    axios bootstrap
npm run dev     # http://localhost:5173

# in src/main.js
import 'bootstrap/dist/css/bootstrap.min.css'
```

## Check that...

- Delete HelloWorld.vue, style.css and assets/
- Bootstrap comes from npm – no CDN links
- Flask is running in another terminal
- The page opens at localhost:5173

# A component with the Options API

```
export default {
  name: 'LoginView',
  data() {                // the page's own values
    return { form: { email: '', password: '' }, error: '' }
  },
  methods: {
    async login() {
      this.error = ''
      try {
        await this.$store.dispatch('login', this.form)
        if (this.$store.getters.isAdmin) {
          this.$router.push('/admin')
        } else {
          this.$router.push('/dashboard')
        }
      } catch (error) {
        this.error = errorText(error)
      }
    },
  },
}
```



**One object**

data, computed, mounted, methods – each in its own section



**this**

this.form, this.$store, this.$router, this.$route



**No arrow methods**

write login() {} so this is the component

# api.js: one axios object for every page

```
const api = axios.create({
  baseURL: 'http://localhost:5000/api',
})

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers['Authentication-Token'] = token
  return config
})

api.interceptors.response.use(
  (response) => response,
  (error) => {
    const status = error.response && error.response.status
    if (status === 401 && localStorage.getItem('token')) {
      localStorage.clear()
      window.location.href = '/login'
    }
    return Promise.reject(error)
  },
)
```



**Before each request**

adds the Authentication-Token header



**After each response**

401 = token not valid → clear it → login page



**In a page**

import api from '../api' then api.get('/me')

# Vuex: one-way data flow




One file: store/index.js. No modules, no namespaces. Getters isLoggedIn and isAdmin are worked out from the state.

# store/index.js: state, getters, mutations

```
state: {
  token: localStorage.getItem('token'),
  user: JSON.parse(localStorage.getItem('user')),
  message: null,                  // { type, text }
},
getters: {
  isLoggedIn(state) { return state.token !== null },
  isAdmin(state) {
    return state.user !== null &&
           state.user.roles.includes('admin')
  },
},
mutations: {
  setLogin(state, data) {
    state.token = data.token
    state.user = data.user
  },
  clearLogin(state) { state.token = null; state.user = null },
  setMessage(state, message) { state.message = message },
},
```



**state**

read back from localStorage, so a refresh keeps you logged in



**getters**

answers worked out from the state



**mutations**

the only place that changes state – no API calls here

# store/index.js: actions

```
actions: {
  async login(context, form) {
    const response = await api.post('/login', form)
    const data = response.data          // { token, user }
    localStorage.setItem('token', data.token)
    localStorage.setItem('user', JSON.stringify(data.user))
    context.commit('setLogin', data)
  },
  async logout(context) {
    try {
      await api.post('/logout')
    } finally {
      localStorage.clear()
      context.commit('clearLogin')
    }
  },
  showMessage(context, message) {
    context.commit('setMessage', message)
    setTimeout(() => context.commit('setMessage', null), 3000)
  },
},
```



**Call the API**

actions may be async and talk to Flask



**localStorage**

remember the login after a refresh



**commit**

hand the result to a mutation

# Using the store from any page

| I want to... | Write |
|---|---|
| read the user | `this.$store.state.user` |
| check the role | `this.$store.getters.isAdmin` |
| log in / log out | `this.$store.dispatch('login', this.form)` |
| show a message | `this.$store.dispatch('showMessage', { type: 'success', text: 'Saved' })` |
| use it in the template | `computed: { user() { return this.$store.state.user } }` |

Register does not use the store: it calls api.post(’/register’) directly, because nothing has to be remembered afterwards.

# Routes and the guard

frontend/src/router/index.js

```
const routes = [
  { path: '/', redirect: '/login' },
  { path: '/login', component: LoginView },
  { path: '/admin', component: AdminDashboard, meta: { role: 'admin' } },
  { path: '/dashboard', component: UserDashboard, meta: { role: 'user' } },
  { path: '/:pathMatch(.*)*', component: NotFound },
]

router.beforeEach((to) => {          // runs before every page change
  const isLoggedIn = store.getters.isLoggedIn
  const isAdmin = store.getters.isAdmin
  const home = isAdmin ? '/admin' : '/dashboard'

  if (to.path === '/login' && isLoggedIn) return home
  if (to.meta.role && !isLoggedIn) return '/login'
  if (to.meta.role === 'admin' && !isAdmin) return home
  if (to.meta.role === 'user' && isAdmin) return home
})
```

# Route guard: who may open a page




Guards are for user experience only – the real security is the token and role check in Flask (Day 2).

# Files built today

| File | What & why |
|---|---|
| `src/main.js` | Bootstrap CSS, use(store).use(router), mount |
| `src/api.js` | axios object, 2 interceptors, errorText() |
| `src/store/index.js` | state, getters, mutations, actions – one file |
| `src/router/index.js` | routes with meta.role + beforeEach guard |
| `components/NavBar, FlashMessage` | user name + logout; message box |
| `views/LoginView, RegisterView` | forms with v-model + HTML5 validation |

# Common errors

| Error | Fix |
| --- | --- |
| CORS error in console | Flask not running, or FRONTEND_ORIGIN ≠ http://localhost:5173 |
| [vuex] unknown action type | the name in dispatch('...') must match a name under actions |
| Cannot read properties of null (reading 'full_name') | user is null after logout – show it only with v-if |
| Cannot read properties of undefined | a method was an arrow function – use login() { ... } |
| Logged out after refresh | the login action must save token and user in localStorage |

# Recap

- A Vue SPA talks to Flask only through JSON
- api.js adds the token to every request
- One small store: dispatch → action → commit → mutation → state
- The router guard sends each role to its own pages

**Next:** Day 5 – all admin and user screens. Each page loads its own data; the store does not grow.